

백종현

사용자의 관점에서 끊임없이 고민하고, 안정적인 서비스로 답하는 개발자

gkdisrha2020@gmail.com · github.com/corinB

요약

예매 시스템 개발 중 부하 테스트를 통해 대기열 후순위 사용자의 결제 진입이 지연되는 병목을 발견했습니다. 기존 구조는 재고가 여러 장 복구되어도 선두 사용자 한 명만 결제 단계로 이동하는 방식이었고, 이를 재고 수와 대기열 길이에 따라 결제 가능 인원을 동적으로 조정하는 구조로 개선했습니다.

자기소개

[문제 해결] 사용자의 불편에서 출발해 원인을 끝까지 파고드는 개발자

저는 단순히 기능을 구현하는 것보다, 사용자가 겪는 불편이 어디에서 시작되는지 파악하는 일을 중요하게 생각합니다. 예매 서비스 개발 당시 부하 테스트를 진행하던 중 대기열 후순위 사용자의 결제 진입이 지연되는 문제를 발견했고, 이를 단순한 성능 저하가 아니라 결제 처리 흐름에서 발생하는 구조적 병목으로 분석했습니다. 환불·취소로 재고가 한꺼번에 복구되는 상황에서 결제 가능 인원을 동적으로 조정한 결과, 대기열 소진 시간을 3.2초에서 2.0초로 약 36% 단축했습니다.

[설계 역량] 최악의 사용자 경험을 막기 위해 먼저 구조를 고민하는 개발자

여러 프로젝트를 경험하며 초기 설계의 작은 빈틈이 사용자에게 큰 불편으로 이어질 수 있다는 것을 배웠습니다. 특히 예매 시스템에서는 사용자가 결제까지 완료했음에도 티켓이 발급되지 않는 상황을 가장 심각한 사용자 이탈 요인으로 보았습니다. 이를 방지하기 위해 명세 단계부터 “결제 성공 시 티켓 발급, 결제 실패 시 예매 실패 처리” 흐름을 명확히 정의했습니다.

[가치 창출] 제한된 조건 안에서 제품의 핵심 가치를 지키는 개발자

CineStream 진행 중 일정 압박과 CDN 적용의 기술적 부담으로 스트리밍 기능을 제외하자는 의견이 있었습니다. 하지만 스트리밍은 사용자가 서비스의 가치를 직접 경험하는 핵심 기능이라고 판단했습니다. 서비스 대상을 국내 사용자로 한정하고 CDN 없이 HLS 기반 스트리밍을 적용해 복잡도를 낮췄고, 개발 리스크를 줄이면서도 핵심 기능을 기한 내 구현할 수 있었습니다.

프로젝트

CineStream - 실시간 영화 상영 및 예매 플랫폼

팀프로젝트 5인 · 2026.03 ~ 2026.05

PO·팀장 · ticket-service / streaming-service 담당 · Java 21 / Spring Boot 4.0.4 / Redis / Kafka / Quartz / HLS

I. 티켓팅과 라이브 시청을 하나의 시간 흐름으로 연결했습니다

CineStream은 영화 티켓을 구매하고 약속된 시각에 함께 시청하는 서비스를 9개 Spring Boot 마이크로서비스로 구성한 프로젝트입니다. 저는 ticket-service에서 CART, IN_PROGRESSING, TICKETING, LOBBY, STREAMING, FINISH로 이어지는 스케줄 라이프사이클을 Quartz Job으로 연결했고, streaming-service에서는 시청 권한과 HLS/STOMP 세션 흐름을 구현했습니다.

II. 결제 정합성과 대기열 병목을 함께 다뤘습니다

티켓팅 단계에서는 Redis stock, queue, paying 키로 잔여 좌석과 대기열, 결제 진행 인원을 분리했습니다. 외부 user-service 쿠키 차감 HTTP 호출 후 DB 트랜잭션이 실패할 수 있는 구간에는 트랜잭션 종료 콜백으로 환불 보상을 등록했습니다. 오픈런 부하 테스트에서는 재고가 여러 장 복구되어도 한 명씩만 결제 단계로 넘기는 구조의 병목을 확인했고, 재고 수와 대기열 길이를 기준으로 결제 가능 인원을 동적으로 여는 window 방식을 적용했습니다.

III. 환불 burst 상황의 대기열 소진 시간을 검증했습니다

k6 부하 테스트와 별도 환불 시나리오로 stock 100이 동시에 복구되는 상황을 재현했습니다. 대기열을 한 명씩 처리할 때는 소진 시간이 3,152 ms였고, window 방식은 2,017 ms로 측정되어 같은 조건에서 약 36% 빠른 결과를 확인했습니다. 일반 매진 흐름에서는 window 방식의 이득이 작고 p95 지연이 59.83 ms 대 44.88 ms로 더 높아지는 한계도 함께 기록했습니다.

IV. HLS 기반 스트리밍을 제한된 범위 안에서 구현했습니다

일정 압박과 CDN 적용 부담으로 스트리밍 기능 제외 의견이 있었지만, 서비스의 핵심 경험이라고 판단했습니다. 국내 사용자 대상, 단일 노드, CDN 없는 HLS 서버로 범위를 줄였고, 단일 sessionToken을 HLS 쿼리와 STOMP CONNECT 헤더에서 함께 검증해 시청과 채팅 접근을 제어했습니다.

동네마켓 - 지역 상권 기반 커머스 및 배송 플랫폼

팀프로젝트 6인 · 2026.01 ~ 2026.02

PO·팀장 · 라이더 위치 관리 / 배차 흐름 담당 · Java 21 / Spring Boot 3.5.10 / PostGIS / Redis / SSE / Spring Batch

I. 네 역할의 주문·배송 흐름을 하나의 서비스로 구성했습니다

동네마켓은 고객, 마트, 라이더, 관리자가 함께 사용하는 동네 배달 플랫폼입니다. 고객은 장바구니·주문·결제·배달 추적을 사용하고, 마트는 주문 접수와 상품 관리를 처리하며, 라이더는 배달 요청과 위치 공유를 담당합니다. 저는 PO·팀장으로 요구사항을 정리하고 일정과 역할을 조율했습니다.

II. 실시간 위치 데이터와 영속 데이터를 분리했습니다

실시간 라이더 위치 추적 기능 부하 테스트 중 HikariCP 커넥션 풀 고갈 오류를 발견했습니다. 라이더 위치 정보는 장기 보존 데이터가 아니라 짧은 주기로 갱신되고 빠르게 조회되어야 하는 상태성 데이터라고 판단했습니다. 이에 실시간 위치 정보를 DB에서 Redis GEO로 이전해 빈번하게 변경되는 데이터와 PostgreSQL/PostGIS 영속 데이터를 분리했습니다.

III. 배차 경합과 알림 흐름을 분리해 다뤘습니다

같은 배달 요청에 복수 라이더가 동시에 수락하면 중복 배정이 발생할 수 있어 Redis SETNX 기반 분산 락을 적용했습니다. 주문 접수, 배달 배정, 알림 저장은 Spring Events와 SSE, Redis Pub/Sub로 분리해 핵심 주문 로직과 부수 작업의 결합도를 낮췄습니다. Spring Batch 5 기반 정산 흐름도 함께 구성해 마트·라이더 정산을 재실행 가능한 구조로 다뤘습니다.

IV. 문서와 프로토타입으로 팀의 해석 차이를 줄였습니다

짧은 일정 속에서 요구사항이 잘못 해석되는 것을 막기 위해 명세 단계에서 프로토타입 UI를 빠르게 구현했습니다. 데일리 스크럼에서는 PostGIS 반경 조회 태스크의 병목을 확인하고 관련 작업 전환을 제안했습니다. 이를 통해 팀원이 남은 핵심 기능에 집중할 수 있도록 작업 흐름을 조정했습니다.

교육

대구대학교 · 졸업 · 컴퓨터공학

2020.03 ~ 2026.02

대구대학교 · 졸업 · 전자정보

2024.03 ~ 2026.02

대외활동

제13회 IT/공학계열 대학생 작품 경진대회 · 대구대학교 · 최우수상

2025

2024 Global Capstone Design Project · 영남대학교 · Innovation Award

2024

2023학년도 여름새싹캠프 대학생 멘토 · 네이버커넥트 · 교육봉사

2023

포트폴리오

GitHub · github.com/corinB

대외활동 증빙 · [Academic Evidence Portfolio](#)

기술 스택

Backend

Java · Spring Boot · Kafka · Redis

Infra

AWS · Docker · Kubernetes · GitHub Actions

Data

SQL · PostgreSQL · PostGIS · Elasticsearch · QueryDSL